Report (Assignment 4)

George Boufis

This project implements a complete Question Answering (QA) system based on a custom Knowledge Graph (KG) built in Neo4j. The system combines rule-based intent detection, semantic fallback using embeddings, Cypher query execution, and a RAG-style context retrieval mechanism. It also includes full evaluation on clean and adversarial datasets, along with robustness scoring. Below is a summary of the work completed.

The project begins with the construction of a Neo4j Knowledge Graph containing all relevant entities: apartments, amenities, rules, roommates, and owners. Each apartment contains properties such as rent and location, while relationships such as HAS_AMENITY, HAS_RULE, LIVES_IN, and OWNS define the structure of the graph. This KG serves as the single source of truth for the QA system.

A hybrid intent detection module was implemented. First, a rule-based detector identifies intents using keywords and regular expressions. It supports queries about rent thresholds, amenities of an apartment, rules of an apartment, and roommates. If the rule-based approach fails, a semantic fallback is applied using embeddings and cosine similarity. This allows the system to recognise paraphrased or noisy questions by comparing them with predefined template examples.

Once an intent is detected, parameters such as apartmentId or numerical rent limits are extracted through regular expressions. Every intent corresponds to a specific Cypher query that retrieves structured results from Neo4j. The QA pipeline integrates all components into a unified function (run_question), which performs intent detection, parameter extraction, Cypher execution, and, when needed, RAG-style fallback based on keyword-driven triple retrieval. This ensures that even when no clear intent is detected, the system can still provide partially relevant information from the Knowledge Graph.

To support fair evaluation, an answer-normalisation layer was developed. It converts all answer formats—strings, dicts, lists—into comparable lowercase strings by extracting meaningful values such as apartment IDs, amenity names, rule descriptions, or roommate names. This ensures that the accuracy evaluation is robust to formatting differences.

Two evaluation datasets were created. The clean dataset contains well-formed queries that match expected user phrasing. The adversarial dataset includes paraphrased questions, noise words, grammatical mistakes, synonyms, and ambiguous formulations. These datasets allow the system to be evaluated both under standard and adverse conditions.

The evaluation process uses a compute_accuracy function that runs each question through the QA pipeline, normalises the answer, and compares it with the expected output. From this, three metrics are calculated: Clean Accuracy, Adversarial Accuracy, and a Robustness Score defined as the ratio of adversarial accuracy to clean accuracy.

The final results obtained by the system are:
Clean Accuracy = **0.80**
Adversarial Accuracy = **0.3333**
Robustness Score = **0.4167**

These results indicate strong performance on clean questions but limited resilience under adversarial conditions. The semantic fallback improves flexibility, yet parameter extraction and intent recognition become less reliable when queries deviate significantly from expected phrasing. Overall, the system achieves a realistic performance profile for a hybrid Knowledge-Graph-centric QA model.

In summary, the project successfully delivers a functional KG-based QA system that integrates rule-based detection, semantic similarity, Cypher querying, context retrieval, answer normalisation, and quantitative evaluation. It provides a solid foundation for future improvements such as enhanced semantic parsing, broader intent coverage, and stronger adversarial robustness.